

Equipo B: Lozano Morales José Alejandro, Sánchez Pilaloe Andy Paul, Urbina Romero Isaías Abraham

Fecha: 18/06/2026

Título: Modelado de Comunicación entre Procesos Distribuidos

1. Escenario seleccionado

El escenario seleccionado corresponde a un sistema bancario distribuido. Este tipo de sistema requiere la comunicación entre varios procesos independientes, tales como el cliente bancario, el servidor de autenticación, el servicio de cuentas, el servicio de transferencias, el servicio de auditoría, el servicio de notificaciones y la base de datos bancaria.

La elección de este escenario se justifica porque las operaciones bancarias necesitan alta confiabilidad, trazabilidad y disponibilidad. Por esta razón, no todas las comunicaciones usan el mismo mecanismo: algunas requieren entrega garantizada, mientras que otras priorizan rapidez o desacoplamiento entre servicios.

2. Diagrama en Draw.io

El diagrama de comunicación entre procesos fue elaborado en Draw.io y representa los principales componentes del sistema bancario distribuido, así como los mecanismos de comunicación utilizados entre ellos.

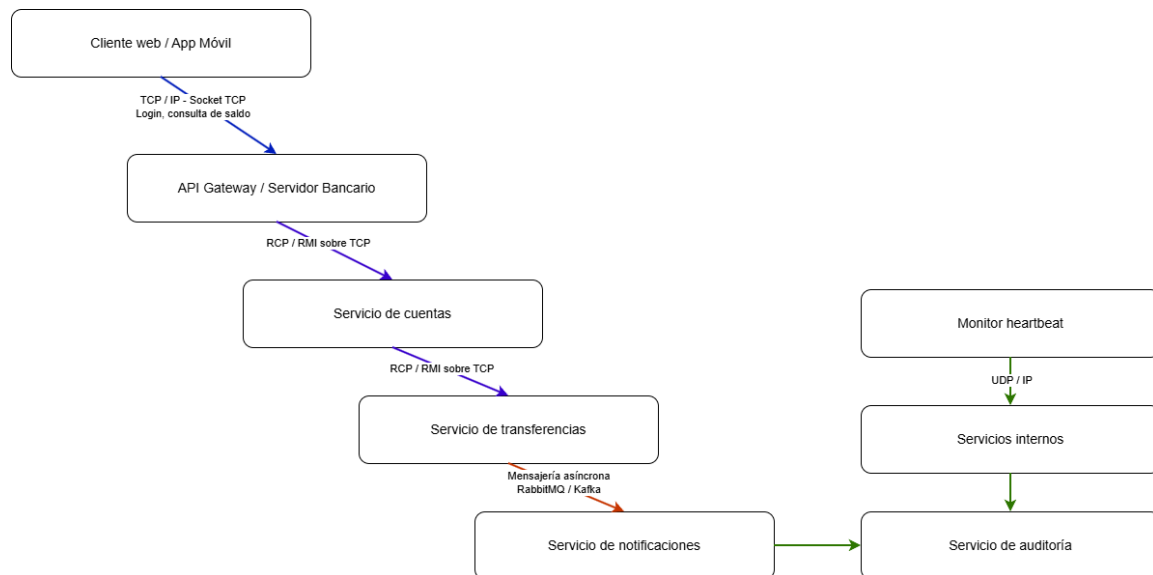


Figura 1: Diagrama de comunicación entre procesos del sistema bancario distribuido. Fuente: elaboración propia en Draw.io.

3. Descripción del modelo de comunicación

El cliente bancario se comunica con el servidor principal mediante sockets TCP, debido a que las operaciones como inicio de sesión, consulta de saldo y transferencias requieren una conexión confiable, ordenada y con confirmación de respuesta. En aplicaciones distribuidas, el rastreo de dependencias entre procesos mediante sockets permite comprender cómo los componentes intercambian información a través de conexiones de red [1].

Entre los servicios internos se propone el uso de RPC o RMI sobre TCP para operaciones que requieren invocación directa de funciones remotas, como validar una cuenta, consultar el saldo disponible o confirmar una transferencia. Los mecanismos RPC permiten coordinar nodos distribuidos y pueden optimizarse para mejorar rendimiento y tolerancia a fallos [2].

Para el registro de eventos y notificaciones se propone mensajería asíncrona mediante RabbitMQ o Kafka, ya que el sistema no debe detener una operación bancaria mientras se registra una auditoría o se envía una notificación al usuario. Este enfoque coincide con arquitecturas basadas en colas, donde un intermediario de mensajes permite evitar bloqueos y conflictos durante el acceso simultáneo a servicios [3].

Finalmente, para el monitoreo de disponibilidad se propone UDP mediante Datagram-Socket, ya que los mensajes de *heartbeat* son ligeros, rápidos y pueden tolerar la pérdida ocasional de algún paquete sin afectar directamente las transacciones bancarias. El uso combinado de TCP y UDP permite separar comunicaciones confiables de mensajes rápidos o tolerantes a pérdida, como ocurre en redes distribuidas con distintos tipos de tráfico [4].

El almacenamiento y procesamiento distribuido de información crítica requiere componentes con alta disponibilidad, escalabilidad y bajo acoplamiento, características presentes en centros de datos distribuidos basados en middleware orientado a mensajes [5].

4. Tabla de comunicación entre procesos

Cuadro 1: Comunicación entre procesos del sistema bancario distribuido

Origen	Destino	Tipo de comunicación	Protocolo / Middleware	Justificación
Cliente bancario	API Gateway / Servidor bancario	Socket TCP	TCP/IP – Java Socket	Permite comunicación confiable para login, consultas y transferencias.
API Gateway	Servicio de cuentas	RPC / RMI	TCP – Java RMI	Facilita la invocación de operaciones remotas como consultar saldo o validar cuenta.

Origen	Destino	Tipo de comunicación	Protocolo Middleware	Justificación
Servicio de cuentas	Base de datos bancaria	Comunicación cliente-servidor	TCP/IP	Garantiza intercambio confiable de datos críticos.
Servicio de transferencias	Servicio de auditoría	Mensajería asíncrona	RabbitMQ o Kafka	Permite registrar eventos sin bloquear la operación principal.
Servicio de transferencias	Servicio de notificaciones	Mensajería asíncrona	RabbitMQ o Kafka	Desacopla el envío de notificaciones del proceso transaccional.
Monitor de disponibilidad	Servicios internos	Socket UDP	UDP/IP – DatagramSocket	Permite enviar <i>heartbeats</i> rápidos para verificar si los servicios siguen activos.

5. Justificación general

La arquitectura propuesta combina distintos mecanismos de comunicación según la necesidad de cada interacción. TCP se usa en operaciones críticas porque ofrece entrega ordenada y confiable. UDP se reserva para monitoreo, donde la velocidad es más importante que la garantía absoluta de entrega. RPC/RMI se aplica para llamadas directas entre servicios internos, mientras que la mensajería asíncrona permite desacoplar eventos secundarios como auditoría y notificaciones.

De esta manera, el modelo no solo representa la comunicación entre procesos, sino que también relaciona cada tecnología con una necesidad real del sistema bancario distribuido.

6. Notas de presentación

Buenos días. Nuestro equipo presenta el modelado de comunicación entre procesos distribuidos aplicado a un sistema bancario.

El escenario seleccionado es un sistema bancario porque permite representar operaciones críticas como autenticación, consulta de saldo, transferencias, auditoría y notificaciones. En este tipo de sistema no todas las comunicaciones tienen la misma prioridad: algunas requieren confiabilidad total, mientras que otras necesitan rapidez o desacoplamiento.

En el diagrama se observa primero al cliente bancario, que puede ser una aplicación móvil o web. Este cliente se comunica con el servidor principal mediante sockets TCP, porque operaciones como login, consulta de saldo y transferencias necesitan entrega confiable, ordenada y con respuesta del servidor.

Luego, el servidor principal se comunica con servicios internos como cuentas y transferencias mediante RPC o RMI sobre TCP. Esto permite que un proceso invoque funciones re-

motas, por ejemplo `consultarSaldo()`, `validarCuenta()` o `procesarTransferencia()`, como si fueran métodos locales.

Para procesos secundarios como auditoría y notificaciones se propone mensajería asíncrona mediante RabbitMQ o Kafka. Esto evita que la transferencia se bloquee mientras se registra un evento o se envía una notificación al usuario.

Finalmente, se incluye un monitor de disponibilidad que usa UDP para enviar mensajes de *heartbeat* a los servicios internos. Se usa UDP porque estos mensajes son rápidos, pequeños y pueden tolerar la pérdida ocasional de algún paquete.

En conclusión, el modelo combina TCP, RPC/RMI, mensajería y UDP según la necesidad de cada comunicación. TCP se usa para operaciones críticas, UDP para monitoreo rápido, RPC/RMI para llamadas internas y mensajería para desacoplar procesos secundarios.

7. Pregunta crítica

Pregunta: ¿Por qué no usar TCP para todo el sistema si es más confiable que UDP?

Respuesta: No se usa TCP para todo porque no todas las comunicaciones tienen la misma necesidad. En operaciones críticas como transferencias o consultas bancarias sí es necesario usar TCP porque garantiza entrega y orden. Sin embargo, para mensajes de *heartbeat* o monitoreo, UDP es suficiente porque son paquetes pequeños, frecuentes y toleran pérdidas ocasionales. Si un *heartbeat* se pierde, el siguiente puede confirmar si el servicio sigue activo.

8. Anexos

Las siguientes capturas corresponden a la evidencia técnica de implementación de sockets TCP y UDP en Java. Esta evidencia complementa el modelo de comunicación propuesto, pero el entregable principal del taller es el diagrama de comunicación entre procesos distribuidos.

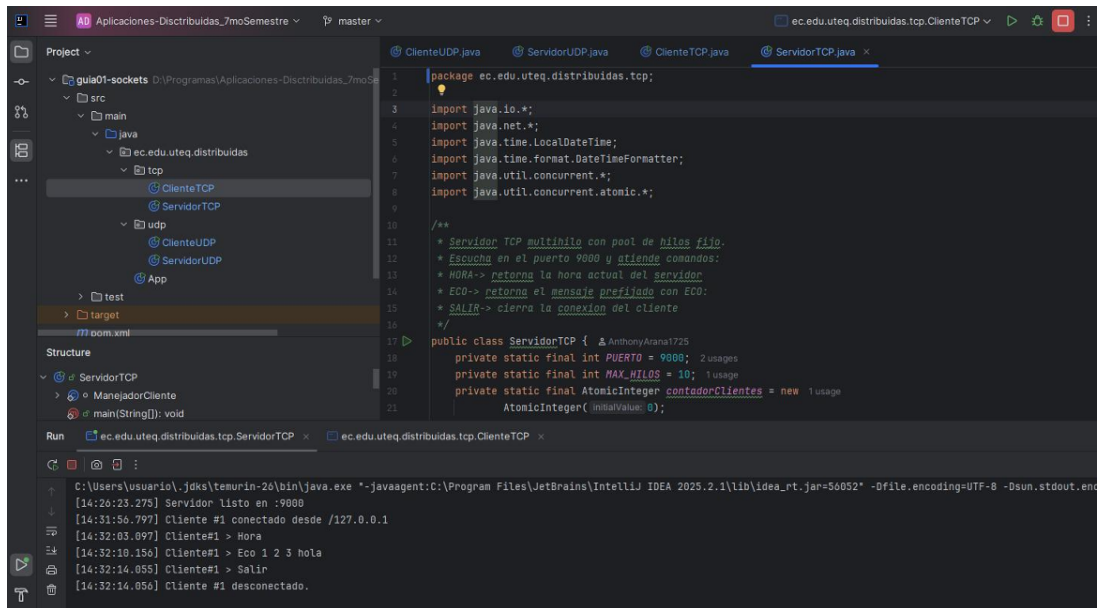


Figura 2: Ejecución del servidor TCP multihilo en IntelliJ IDEA. Fuente: elaboración propia.

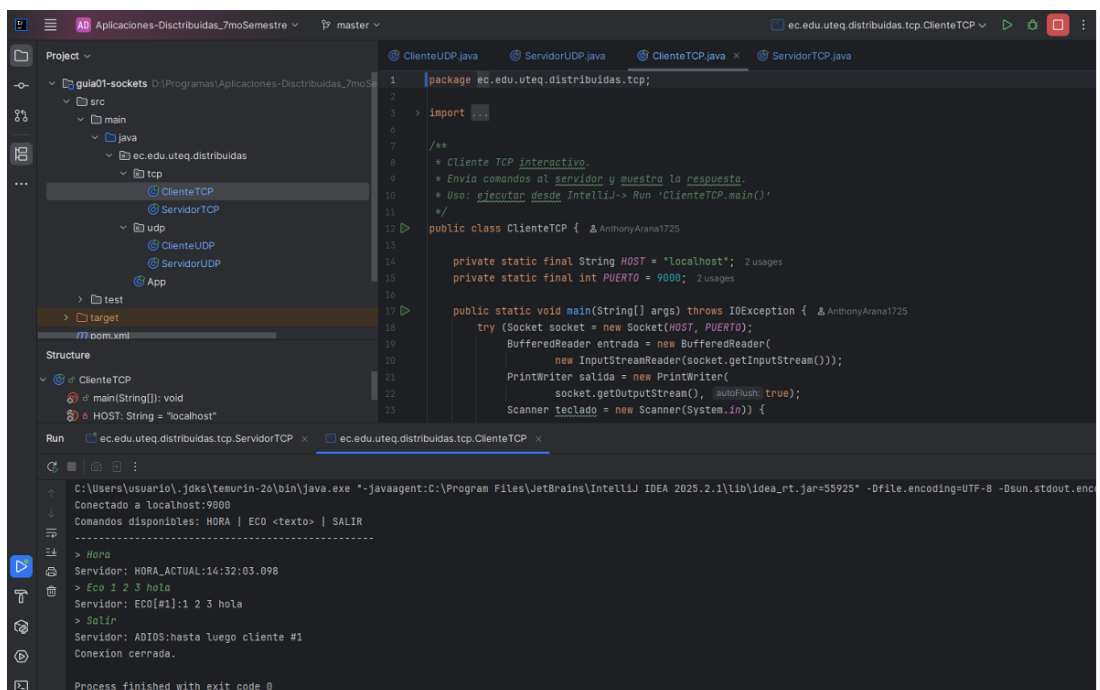


Figura 3: Ejecución del cliente TCP conectado al servidor en localhost mediante el puerto 9000. Fuente: elaboración propia.

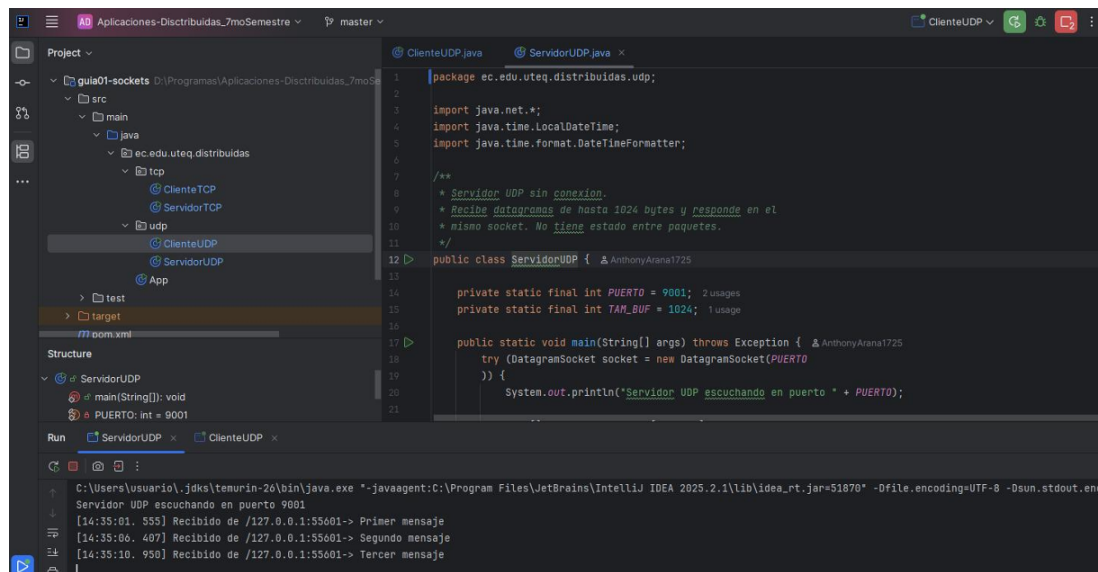


Figura 4: Ejecución del servidor UDP recibiendo datagramas mediante el puerto 9001. Fuente: elaboración propia.

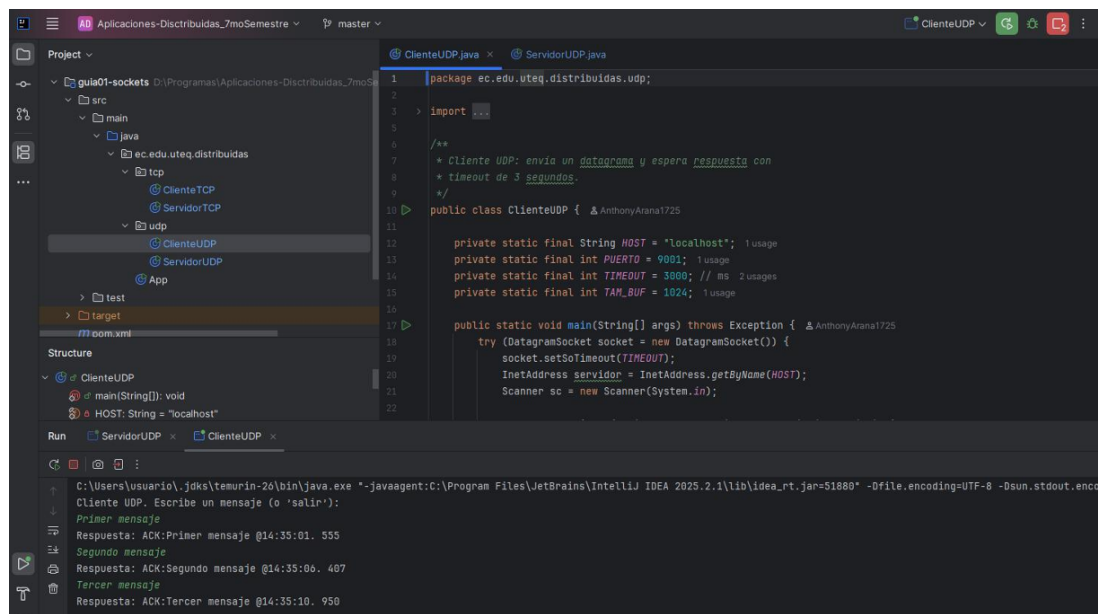


Figura 5: Ejecución del cliente UDP enviando mensajes al servidor y recibiendo respuestas ACK con control de *timeout*. Fuente: elaboración propia.

9. Referencias

- [1] Y. Tsubouchi, M. Furukawa y R. Matsumoto, «Transtracer: Socket-Based Tracing of Network Dependencies Among Processes in Distributed Applications,» en *2020 IEEE 44th Annual Computers, Software, and Applications Conference (COMPSAC)*, IEEE, 2020. DOI: 10.1109/COMPSAC48688.2020.00-92.

- [2] J. Zhang, Z. Shu y L. Luo, «An Adaptive RPC Mechanism for Performance and Node Fault Tolerance Optimization in HDFS,» en *2022 IEEE 24th International Conference on High Performance Computing and Communications; 8th International Conference on Data Science and Systems; 20th International Conference on Smart City; 8th International Conference on Dependability in Sensor, Cloud and Big Data Systems and Application*, IEEE, 2022. DOI: 10.1109/HPCC-DSS-SmartCity-DependSys57074.2022.00270.
- [3] H.-Y. Huang, Y.-J. Chen, Y.-P. Chiu, S.-H. Chiu, S.-C. Hsu e Y.-Y. Fanjiang, «Design Asynchronous Microservice Architecture Using Queue Structure and Lightweight Digital Signature,» en *2025 IEEE 14th Global Conference on Consumer Electronics (GCCE)*, IEEE, 2025. DOI: 10.1109/GCCE65946.2025.11274908.
- [4] Y. Li, F. Du, P. Wang e Y. Yin, «A Multi-robot Topic Communication Method Based on TCP and UDP,» en *2020 Chinese Automation Congress (CAC)*, IEEE, 2020. DOI: 10.1109/CAC51589.2020.9327802.
- [5] F. Yang, H. Liu y T. Bi, «A Distributed Data Center for Full-view Synchronous Measurement System Based on OpenPDC/OpenHistorian,» en *2021 IEEE 4th International Electrical and Energy Conference (CIEEC)*, IEEE, 2021. DOI: 10.1109/CIEEC50170.2021.9511078.